

 PYTHON

 GITHUB PORTFOLIO

Simulador de Análise de Empréstimo

Projeto desenvolvido em **Python** para prática de lógica de programação, modelagem de regras de negócio e simulação de análise de crédito.

Uma aplicação interativa que avalia salário, valor solicitado e quantidade de parcelas para determinar a aprovação ou reprovação de um empréstimo — com suporte a perfil **Cliente Premium**.

O que este sistema faz?

O **Simulador de Análise de Empréstimo** é um programa em Python que simula uma análise básica de crédito. Ele coleta dados do cliente, calcula o valor das parcelas e verifica se o comprometimento da renda está dentro do limite aceitável — definido como **até 30% do salário bruto**.

Regra Principal

O valor da parcela mensal não pode ultrapassar **30% da renda bruta** do cliente. Caso ultrapasse, o empréstimo é reprovado automaticamente.

Cliente Premium

Clientes com perfil **Premium** possuem critérios diferenciados, podendo ter acesso a condições especiais de aprovação mesmo em cenários limítrofes.

Saída do Sistema

Ao final, o sistema exhibe: valor da parcela, percentual de comprometimento da renda e o resultado da análise (aprovado ou reprovado).

Fluxo de Execução do Sistema

O diagrama abaixo representa o fluxo completo do programa, desde o início da execução até a exibição do resultado final ao usuário.



- ❏ A decisão de aprovação ou reprovação considera dois fatores: o **limite de 30% da renda** e o **perfil do cliente** (Premium ou Standard).

Passo a Passo do Código

Cada etapa do desenvolvimento segue uma lógica estruturada, garantindo clareza, manutenibilidade e boas práticas de programação em Python.

1	Entrada dos Dados Coleta das informações via <code>input()</code>: nome, salário, valor solicitado e parcelas.
2	Criação das Variáveis Armazenamento dos dados em variáveis com nomes descritivos e semânticos.
3	Conversão de Tipos Uso de <code>float()</code> e <code>int()</code> para converter strings em números.
4	Cálculo da Parcela Divisão do valor solicitado pela quantidade de parcelas: <code>parcela = valor / qtd</code>.
5	Comprometimento da Renda Cálculo do percentual: <code>(parcela / salario) * 100</code>.
6	Regras de Negócio Estrutura condicional <code>if/elif/else</code> aplicando o limite de 30% e verificando perfil Premium.
7	Cliente Premium Verificação via <code>input()</code> com resposta S/N para flexibilizar critérios.
8	Resultado Final Exibição formatada com <code>:.2f</code> mostrando parcela, percentual e decisão final.

Comandos Python e Código Comentado

Comandos Utilizados

`input()`

Lê dados do teclado como string.

`print()`

Exibe informações na tela.

`float()` / `int()`

Converte string para número decimal ou inteiro.

`if` / `elif` / `else`

Estrutura condicional para tomada de decisão.

Operadores `+` `-` `*` `/`

Realizam cálculos aritméticos.

Operadores `<` `>` `==`

Comparam valores e retornam booleano.

Formatação `:.2f`

Exibe floats com 2 casas decimais.

Código Completo Comentado

```
# Simulador de Análise de Empréstimo
# Projeto de portfólio em Python

# --- Entrada de dados ---
nome = input("Nome do cliente: ") # Lê o nome
salario = float(input("Salário bruto: ")) # Converte
para float
valor = float(input("Valor solicitado: ")) # Valor do
empréstimo
parcelas = int(input("Nº de parcelas: ")) # Converte
para int
premium = input("É Premium? (S/N): ").upper()

# --- Cálculos ---
vlr_parcela = valor / parcelas # Valor de cada parcela
comprometimento = (vlr_parcela / salario) * 100 # %
da renda

# --- Exibição dos dados ---
print(f"\nCliente: {nome}")
print(f"Parcela: R$ {vlr_parcela:.2f}")
print(f"Comprometimento:
{comprometimento:.2f}%")

# --- Regras de negócio ---
if comprometimento <= 30:
    print("Aprovado")
```

Conceitos Praticados e Melhorias Futuras

Conceitos Desenvolvidos

- **Lógica de programação** — sequência, decisão e cálculo.
- **Tipos de dados** — string, int, float e conversão.
- **Estruturas condicionais** — if, elif e else.
- **Operadores** — aritméticos, relacionais e lógicos.
- **Modelagem de regras de negócio** — limite de 30% da renda.
- **Formatação de saída** — f-strings e `:.2f`.

Melhorias Futuras

- **Taxas de juros** — simular diferentes cenários de CET.
- **Score de crédito** — integrar avaliação de risco.
- **Histórico do cliente** — considerar inadimplência passada.
- **Banco de dados** — persistir consultas com SQLite.
- **Interface gráfica** — criar GUI com Tkinter ou Flask.
- **API REST** — expor o simulador como serviço web.
- **Geração de contratos** — emitir PDFs com os termos.
- **Dashboard** — visualizar métricas com Matplotlib.

Este projeto consolidou fundamentos essenciais de Python e demonstrou como transformar regras de negócio em código funcional — base sólida para sistemas mais complexos.